

变分自编码器实验手册

第060单元 核对概率目标并观察后验坍塌

本实验包括二维连续数据和 8×8 二值图像，两种观测模型都真正训练。你要完成的不是挑几张好看的生成图，而是检查KL算术、NLL归约、重参数化梯度、坍塌诊断和单因素干预。总计12个模型，每个600步，数据和权重均随包提供，CPU即可完成。

安装与文件入口

在课程仓库根目录按下面顺序执行；单独下载单元时直接进入060目录。先安装学习依赖，构建PDF所需软件单列在build-tools。实验不下载外部数据，不需要GPU、账号或API密钥。安装阶段需要软件仓库连接，安装后可离线重跑。

```
cd units/060
python -m venv .venv
# 按操作系统激活该环境后
python -m pip install -r requirements.txt
python test_experiment.py
python -O test_experiment.py
```

先读outputs/protocol.json，记录潜在维数2、批64、600步、高斯sigma0.25、测试16次MC、种子6001和6002。standard全程beta1，high_beta全程beta100，restore_beta前300步100、后300步1。三者步数相同，不以测试结果提前停止。

打开experiment.ipynb，选择刚安装的Python环境，从头顺序运行。Notebook会创建notebook_replay，保留参考outputs。若想只从脚本运行，使用python experiment.py --out my_replay。不要把自己运行的随机结果覆盖参考权重后再声称“重放完全一致”。

第一站 先用小数值检查目标

数据生成器保存两个任务各768训练、192验证和384测试样本。二维点接近半径1.7的环；图像由带随机像素翻转的横线或竖线构成。图像是真正0/1数组，不是把普通灰度图简单缩放成0至1。验证集随包保留，但本实验不据它选择超参数；所有配置提前确定。

先运行KL手算： $\mu=(1,0)$ 、方差 $=(4,1)$ ，即 $\log\text{var}=(\log 4,0)$ ，预期KL约1.306853。再令 μ 与 $\log\text{var}$ 都为0，预期KL为0。解释第二个例子对应标准差1，而不是标准差0。

```
import torch, math
from experiment import kl_standard, reparameterize
mu = torch.tensor([[1., 0.]])
lv = torch.tensor([[math.log(4), 0.]])
eps = torch.tensor([[.5, -1.]])
print(kl_standard(mu, lv))
print(reparameterize(mu, lv, eps))
```

第二个输出应为(2,-1)。Notebook另对 z_1+z_2 求导，预期 μ 梯度(1,1)， $\log\text{var}$ 梯度(0.5,-0.5)。这是局部梯度校验，不是训练时完整损失的一步梯度。

接着检查高斯常数： $D=2$ 、 $\sigma=1$ 、预测等于输入时每例NLL应为 $\log(2\pi) \approx 1.837877$ 。二值64维零logit时NLL应为 $64\log 2 \approx 44.361420$ 。若你得到0.693147，说明做了每像素平均，而非本课每例像素和。不要为“让数值接近”随意调 β ，先检查定义。

第二站 完整训练与重放

每种数据任务训练两种初始化、三个配置。每步有放回抽64行，并给每例抽二维标准正态 ϵ 。二维解码器输出高斯均值，图像解码器输出logit。编码器与解码器都使用32维Tanh隐藏层。训练目标先逐例加NLL和 β 倍KL，再对批次平均。

```
python experiment.py --out my_replay
```

CPU单线程，12次训练共7200个更新步。运行结束打印每个配置的NLL、KL、负ELBO、 μ 方差与配对打乱NLL差。正常运行不会每步打印；耗时因机器而异。若内存不足，先关闭其他进程，不要悄悄缩减测试样本或删除配置。

检查保存的weights文件共12份，均为普通npz数组。plot_data保存每例测试评估、均值重构、先验样本和训练轨迹。不要把训练轨迹中最后一批的loss当作完整测试负ELBO；两者数据、噪声数量与 β 可能都不同。

```
from test_experiment import main
main("my_replay")
```

重放测试会从每个保存模型重新编码全部384个测试样本，按固定噪声种子做16次MC，检查逐例数组与汇总数字。同时检查数据重新生成一致、12配置齐全、图像样本只含0/1、干预前300步的每10步记录完全一致。缺一个高 β 模型也不能通过。

Notebook会做同样完整重训，不只是读参考JSON。它的输出执行计数必须完整且无错误。随包提供的execute_notebook.py使用顺序in-process IPython内核，说明的是代码格执行，不宣称已自动测试所有浏览器Jupyter交互。

第三站 诊断坍塌并审查干预

对每种任务、每个种子，分别抄下standard、high_beta与restore_beta的KL、负ELBO、mu方差和shuffle_nll_delta。先看KL是否极小，再看mu是否随输入变化，最后看打乱mu与x配对是否改变重构。三项一致比一个模糊图更有说服力。

二维高beta的KL约 $1e-4$ 量级，图像约 $1e-5$ 量级；restore_beta恢复到大于1的量级。这里的数量级来自本数据与尺度，不是可搬到所有VAE的阈值。beta100是人为压力，结论应明确“诱发坍塌”，不要写成标准beta1必然坍塌。

观察第300步之前的两条高beta前缀，它们应逐数相同。后半段只改变beta，网络、优化器状态、批次和噪声序列不改。若另从随机初始化训练一个beta1模型，然后称它“恢复”，就无法回答同一个已坍塌模型是否能重新利用潜变量。

图像恢复配置虽优于high_beta，但负ELBO仍约23.08至23.37，差于标准约19.97至20.63。请保留这种不完全恢复，不把二维结果扩写成两个任务都全面成功。提出解释时可以谈恢复步数和优化路径，但本实验没有分别验证这些原因，因此只能标成假设。

看图前先给图命名

图01是从先验生成并加观测噪声的二维点；图02是测试输入的后验mu解码概率均值；图06区分先验解码概率均值与真正二值抽样。它们不是同一个“生成图片”。用这三个名称描述自己的截图，避免对模糊程度做错误解释。

最后一站 记录限制和排查错误

一段完整报告应包含两种观测模型、nat/例归约、测试MC次数、两个种子、全部配置与单因素干预。明确负ELBO是估计的下界目标，不是精确边缘似然；高beta训练目标不同于最终报告的标准负ELBO；连续二维NLL允许负值。

如果KL明显为负，检查logvar是否被当成log标准差；如果换输入维度后训练行为剧变，检查像素和与平均；如果生成图始终灰度却声称二值样本，检查是否漏了伯努利抽样；如果restore前缀不一致，检查随机数生成器、初始化与更新顺序。

字体方框属于绘图环境问题，安装官方Noto CJK字体再运行make_figures.py。缺依赖时确认Notebook和终端环境一致。不要为绕过检查而删除ValueError或测试；输入保护可以帮助定位定义错误。

完成清单：

- 手算KL约1.306853，重参数化 $z=(2,-1)$ ，梯度核对通过
- 高斯NLL保留归一化常数，伯努利重构项对64维求和
- 普通与-O测试均通过，12模型与12结果完整
- Notebook完整重训到独立目录，参考outputs未覆盖
- 三项诊断支持人为高beta诱发坍塌，恢复前缀逐数一致
- 如实记录图像不完全恢复与两个种子的限制
- 区分均值重构、先验均值、观测抽样和MC负ELBO

最后独立完成十道练习再读答案。能把一个损失写清单位和求和顺序，是学习概率生成模型最重要的起点之一。